Trevor Labat
Student ID: 801340975
GitHub: https://github.com/TlabatCharlotte/Machine-Learning

Homework #7

Problem 1 - Fully Connected Neural Network Architecture

Baseline Fully Connected Neural Network

The CIFAR-10 dataset was used to train a neural network with two hidden layers using ReLU activation functions and an output layer of 10 neurons. The data was split into training and validation sets by asserting/deasserting the train parameter in the CIFAR10 class. The code for this is showcased in Figure 1.1.

```
from google.colab import drive
drive.mount('/content/drive')

from torchvision import datasets
data_path = '/content/drive/MyDrive/Machine Learning/data/'

cifar10 = datasets.CIFAR10(data_path, train=True, download=True)
cifar10_val = datasets.CIFAR10(data_path, train=False, download=True)
```

Figure 1.1 - CIFAR10 configuration

It is then transformed into tensors so that we can get the mean and standard deviation values for normalization. The transformation is also necessary for the PyTorch neural network as it expects tensors for its input; the code for this process can be seen in Figure 1.2. The mean and standard deviation for the RGB values gathered from the aforementioned code are the following:

RGB Mean tensor: ([0.4914, 0.4822, 0.4465])
RGB Standard Deviation tensor: ([0.2470, 0.2435, 0.2616])

```
from torchvision import transforms
to_tensor = transforms.ToTensor()

tensor_cifar10 = datasets.CIFAR10(data_path, train=True, download=False,
                                  transform=to_tensor)

imgs = torch.stack([img_t for img_t, _ in tensor_cifar10], dim=3)
imgs.shape
print(imgs.view(3,-1).mean(dim=1))
print(imgs.view(3,-1).std(dim=1))

transformed_cifar10 = datasets.CIFAR10(
    data_path, train=True, download=False,
    transform=transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465),
                             (0.2470, 0.2435, 0.2616))
]))

transformed_cifar10_val = datasets.CIFAR10(
    data_path, train=False, download=False,
    transform=transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465),
                             (0.2470, 0.2435, 0.2616))
]))
```

Figure 1.2 - PIL to tensor transformation and normalization

The PyTorch DataLoader class is used to create batches that each hold 64 images and their respective labels. This trains the model more accurately than updating the weights after every image, while taking less time and memory than updating the weights once per epoch using all 50000 images.

The neural network is the same classic structure used in the past, 2 hidden layers followed by ReLU activation layers. The difference this time is the LogSoftMax which outputs the logarithmic value of the probabilities used by a new loss function called a negative log likelihood (NLL). The optimizer and its code is implemented the exact same way as the previous homework. The implementation for the neural network, optimizer, and loss function can all be seen in Figure 1.3.

```
model = nn.Sequential(
    nn.Linear(3072, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 10),
    nn.LogSoftmax(dim=1)
)

optimizer = optim.SGD(model.parameters(), lr=1e-2)

loss_fn = nn.NLLLoss()
```

Figure 1.3 - Model, optimizer, and loss function implementation

The forward and backward passes are handled inside a for loop that is nested inside an epoch iterating for loop. This nested loop runs 782 times per epoch because the batch size is 64 (50,000 images / 64 images per batch). This is the main part of the program that takes the longest amount of time, the rest time is taken up by minor calculations for the training/validation accuracy and training loss. This implementation can be seen in Figure 1.4.

```
for epoch in range(n_epochs):
    total_loss = 0
    correct = 0
    total = 0
    for img, label in train_loader:
        batch_size = img.shape[0]
        out = model(img.view(batch_size, -1))
        loss = loss_fn(out, label)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

        _, predicted = torch.max(out, dim=1)
        total += label.shape[0]
        correct += int((predicted == label).sum())

    train_losses.append(total_loss / len(train_loader))
    train_accs.append(correct / total)
    correct = 0
    total = 0

    with torch.no_grad():
      for img, label in val_loader:
        batch_size = img.shape[0]
        out = model(img.view(batch_size, -1))
        _, predicted = torch.max(out, dim=1)
        total += label.shape[0]
        correct += int((predicted == label).sum())
    val_accs.append(correct / total)

    print("Epoch: %d, Loss: %f" % (epoch, total_loss / len(train_loader)))
```

Figure 1.4 - Nested for loop for model training and validation

An epoch value of 80 was chosen for this experiment as it allowed the validation accuracy to converge, or plateau to be more accurate as it appears to be volatile in the plot. The training accuracy was 0.999960 and the validation accuracy was 0.542000 with a training loss of 0.005157. The training loss and training vs evaluation accuracy plots are in Figures 1.5 and 1.6 respectively. When looking at the accuracy we can see that overfitting occurs as the training accuracy gains closer to one while the validation accuracy hovers around 0.50. The total number of parameters trained was 1,841,162 and it took 1954.3s.

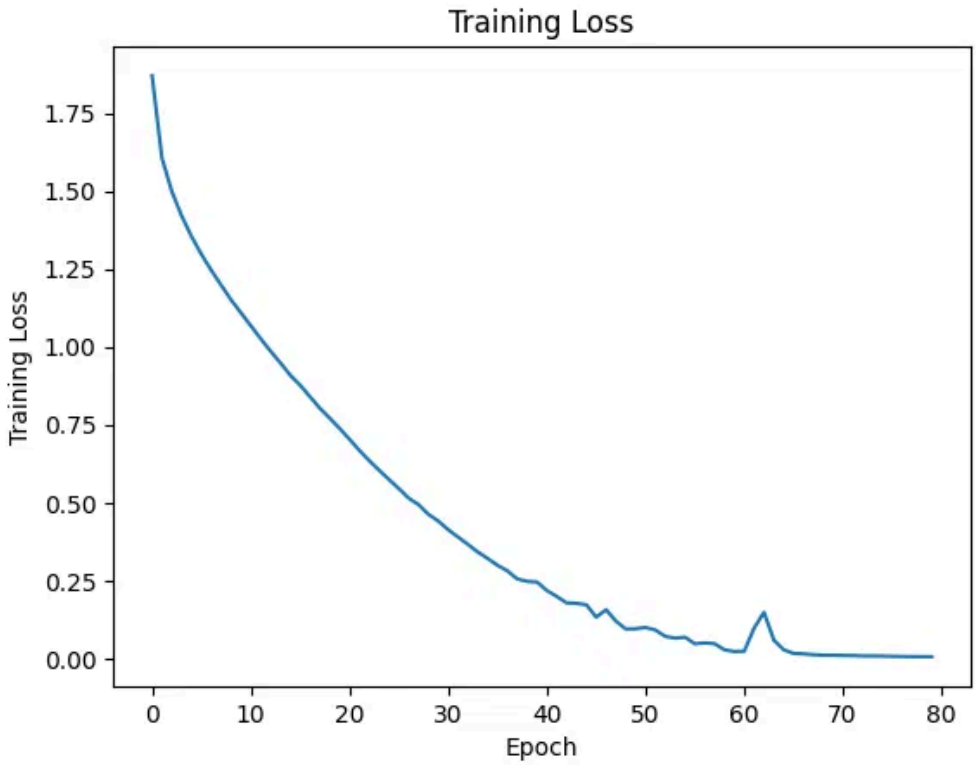


Figure 1.5 - Training loss over 80 epochs

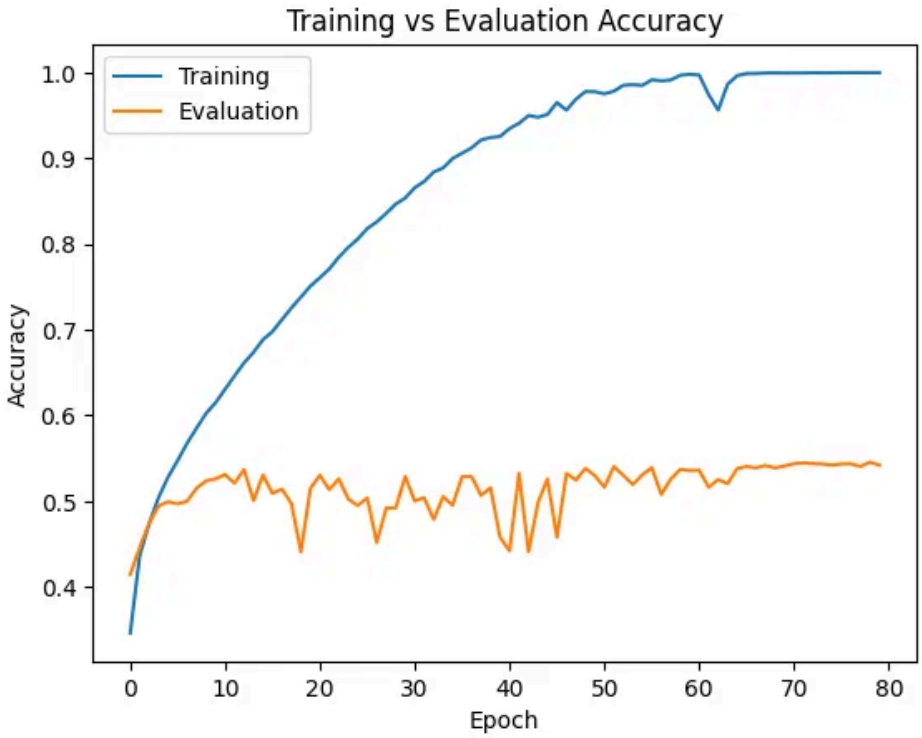


Figure 1.6 - Training and validation accuracy over 80 epochs

Increasing Network Depth

Two additional hidden layers were added to the neural network and the experiment was executed once more. The code for the updated model can be found in Figure 1.6.

```
model = nn.Sequential(
    nn.Linear(3072, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 10),
    nn.LogSoftmax(dim=1)
)
```

Figure 1.6 - Neural network with 4 hidden layers.

The number of epochs were increased to 100 for this experiment to compensate for the slower convergence due to the increased number of hidden layers.

When comparing with the baseline FCNN we can see an increase in the number of parameters to 2,366,474 and an increase in the training time to 2851.3s. The training loss went noticeably down to 0.000285 while the training and validation accuracy stayed the same at 1.000000 and 0.558000 respectively. The plots for the training loss and accuracies can be seen in Figures 1.7 and 1.8. This shows that increasing network depth does not improve classification accuracy by any meaningful amount, both models are textbook overfitting and increasing adding 2 additional layers did not help the classification accuracy.

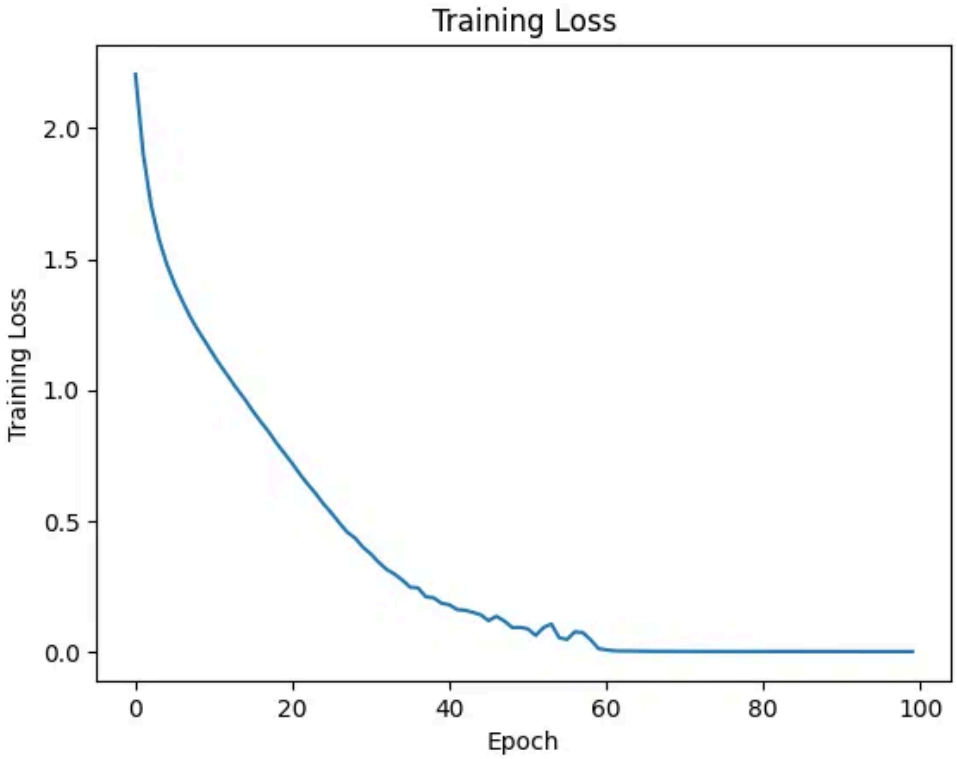


Figure 1.7 - Four hidden layer neural network training loss

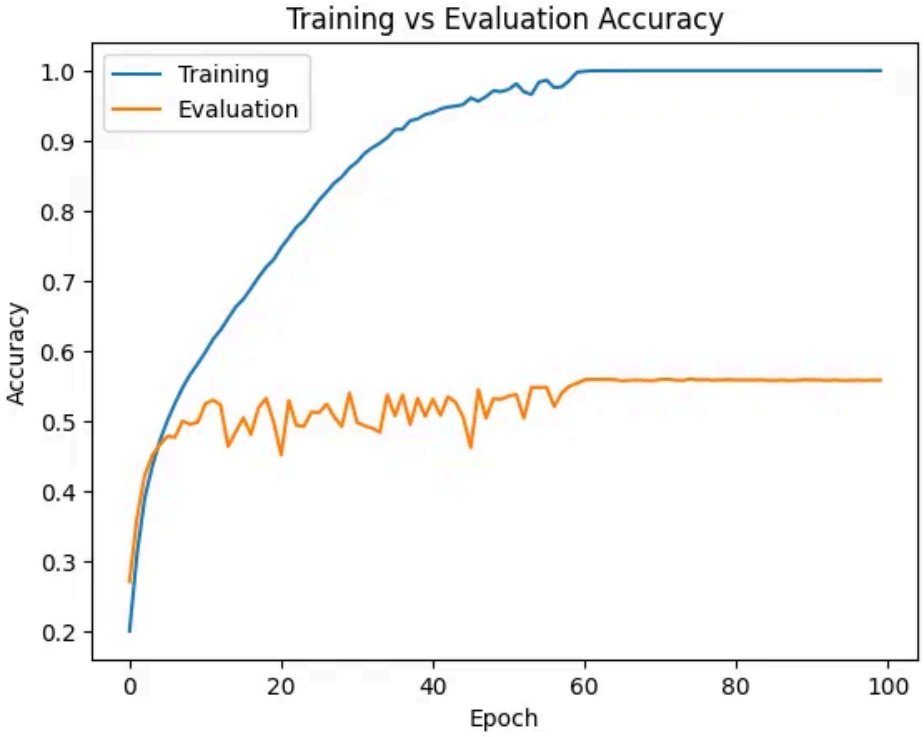


Figure 1.8 - Four hidden layer neural network training and validation accuracy

Problem 2 - Impact of Network Depth and Architecture

Deep Fully Connected Neural Network

The model depth was increased to contain a total of 10 hidden layers, still using the ReLU activation functions with the same number of neurons per layer. The implementation for this is in Figure 2.1. The number of epochs was lower back down to 80 from the previous 100 as the previous experiment converged at 60 epochs.The training time increased even more to 3136.4 and with that it did not converge in its 80 epochs, it could be rerun with 100 epochs but it still would not converge as its training loss was still at 0.642339s. The training and validation accuracy also went down to 0.755080 and 0.434100 while increasing the number of parameters to 3,942,410. The training loss is displayed in Figure 2.2 and the training/validation accuracy is plotted in Figure 2.3.

Strange behavior was observed from epochs 0-40, the loss was stuck at 2.3026. It wasn't until epoch 45 that it started learning at a similar pace to the previous 2 models which needed around 60 epochs to converge.Training deeper models like this is challenging to do in a reasonable amount of time unless you have a bunch of 4090s laying around. The classification performance may have improved over the previous model but not in the maximum number of epochs set by the assignment. Going from 2 to 4 layers helped slightly, but going to 10 made the model slower, bigger, and less accurate within the limit.

```
model = nn.Sequential(
    nn.Linear(3072, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 10),
    nn.LogSoftmax(dim=1)
)
```

Figure 2.1 - Neural network with 10 hidden layers

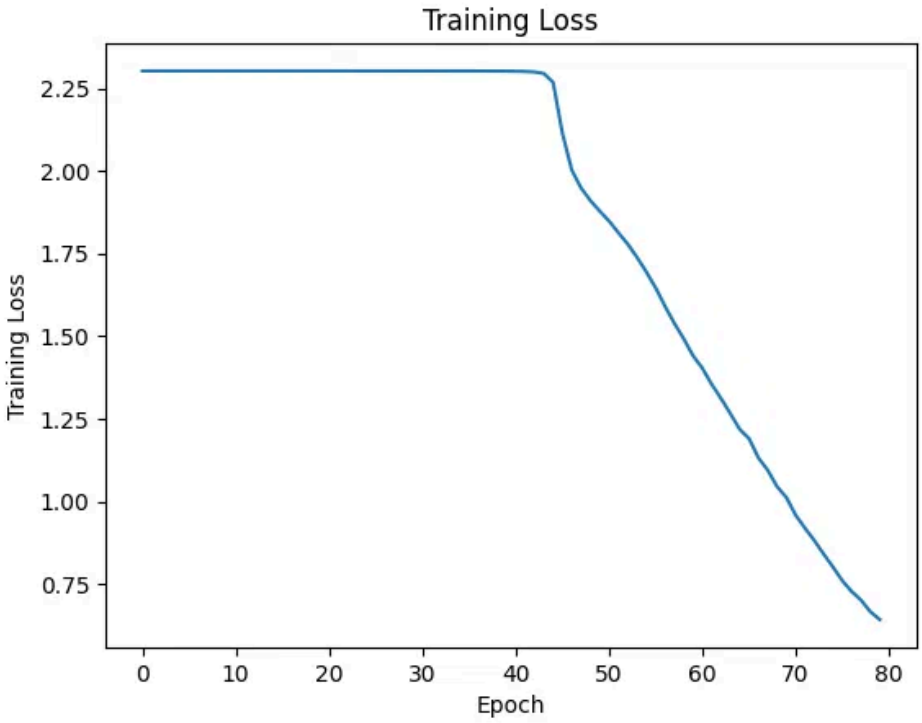


Figure 2.2 - 10 hidden layer neural network training loss

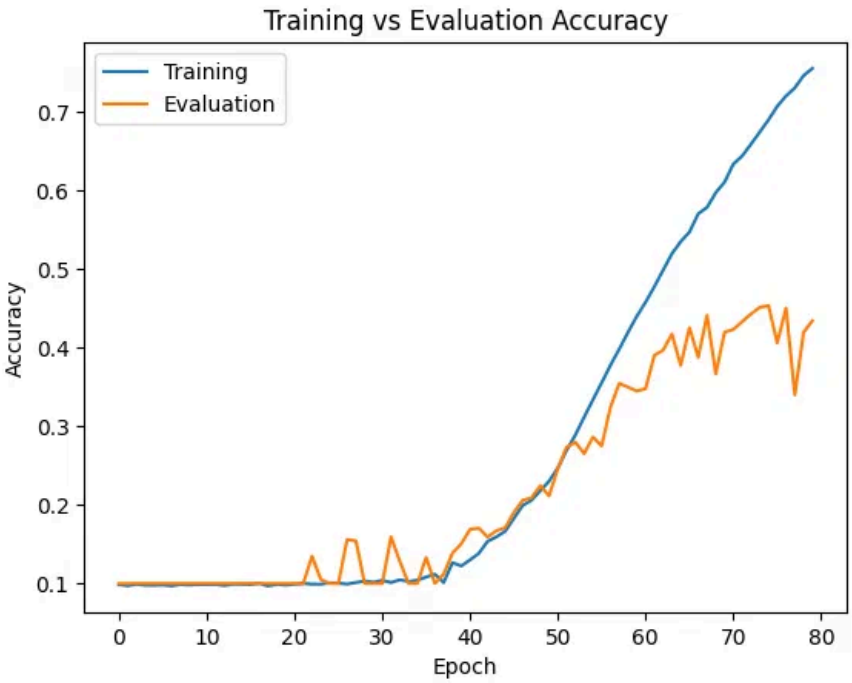


Figure 2.3 - 10 hidden layer neural network training and validation accuracy

Impact of Network Width and Activation Functions

Increasing Width

The number of neurons from the baseline neural network were doubled in each hidden layer. The code for this is in FIgure 2.4. This more than doubled the size of the model making the new parameter count 4,206,602. It also significantly increased the training time to 3301.8s. The validation accuracy slightly increased to 0.558 while the training loss and training accuracy stayed relatively constant. These measurements can be seen in Figures 2.5 and 2.6. This provided results similar to the model with 4 hidden layers, adding a few layers got the same accuracy while using a smaller model and a shorter training time than the model with its width doubled.

```
model = nn.Sequential(
    nn.Linear(3072, 1024),
    nn.ReLU(),
    nn.Linear(1024, 1024),
    nn.ReLU(),
    nn.Linear(1024, 10),
    nn.LogSoftmax(dim=1)
)
```

Figure 2.4 - Baseline Neural network with doubled width

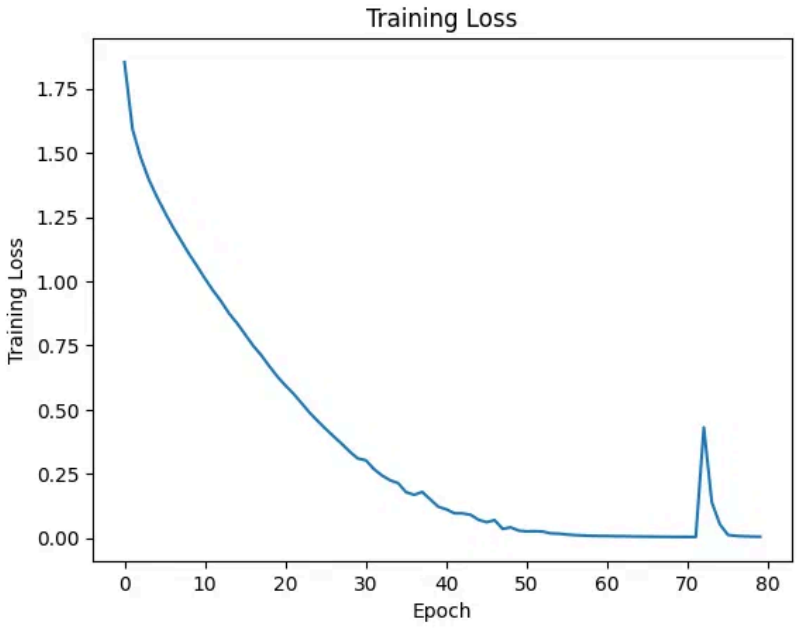


Figure 2.5 - Training loss for baseline neural network with doubled width



Figure 2.6 - Training and Validation Accuracy for baseline neural network with doubled width

Sigmoid Activation

The same baseline neural network was used, this time the activation functions were swapped with Sigmoid. The implementation of this can be seen in Figure 2.7. The training was much slower as the training was definitely not finished in the 80 epoch limit, the final training loss was 1.216. The final training and validation accuracy was 0.581 and 0.4947 respectively. The trainable parameters of course stayed the same at 1,841,162. The plots for this model are in Figures 2.8 and 2.9. The curves are much smoother when compared to models using ReLU, but it is extremely slow so it was nowhere near convergence.

```
model = nn.Sequential(
    nn.Linear(3072, 512),
    nn.Sigmoid(),
    nn.Linear(512, 512),
    nn.Sigmoid(),
    nn.Linear(512, 10),
    nn.LogSoftmax(dim=1)
)
```

Figure 2.7 - Baseline neural network with sigmoid activation function

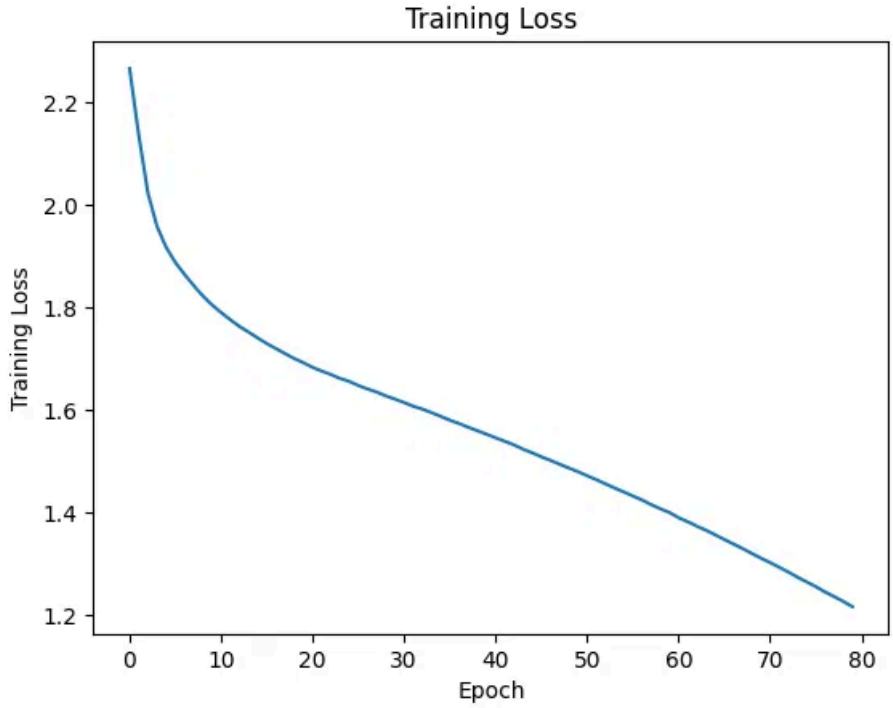


Figure 2.8 - Training loss for baseline neural network with sigmoid activation function



Figure 2.9 - Training and validation accuracy for the baseline neural network using sigmoid activation function

Tanh Activation

The same baseline neural network was used, this time the activation functions were swapped with Tanh. The implementation of this can be seen in Figure 2.10. The model converged but the validation accuracy was the lowest of the converged models as 0.4515. The training loss and accuracy were 0.0198 and 0.998 respectively. The trainable parameters of course remained the same at 1,841,162 but the training time was still a bit slower at 2088.3s. The plots are in Figures 2.11 and 2.12.

```
model = nn.Sequential(
    nn.Linear(3072, 512),
    nn.Tanh(),
    nn.Linear(512, 512),
    nn.Tanh(),
    nn.Linear(512, 10),
    nn.LogSoftmax(dim=1)
)
```

Figure 2.10 - Baseline neural network with Tanh activation function

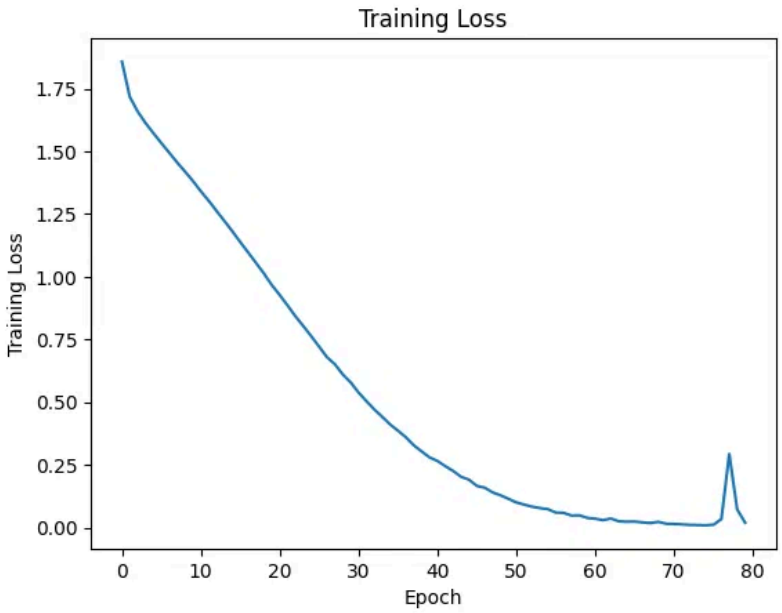


Figure 2.11 - Training loss for the baseline neural network with Tanh activation function

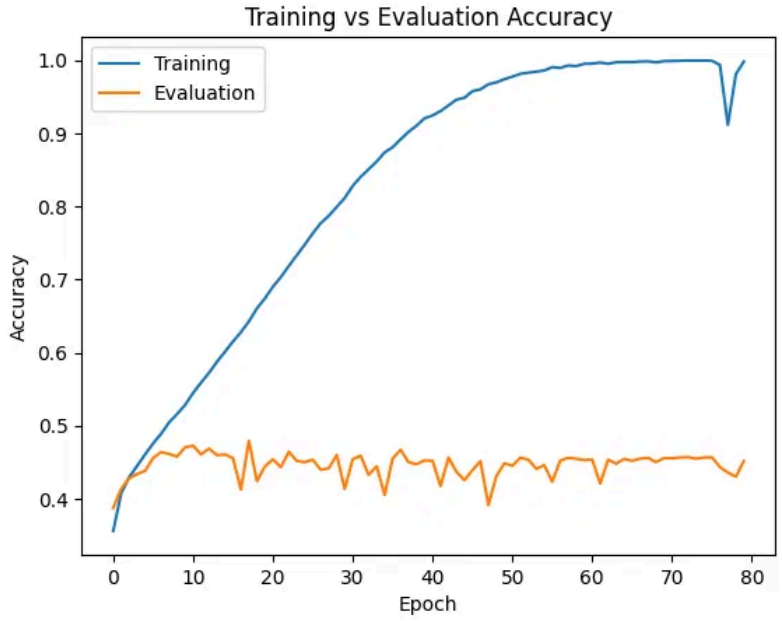


Figure 2.12 - Training and validation accuracy for the baseline neural network with Tanh activation function

The best architecture overall is the 4 hidden layer neural network using the ReLU activation function, it is tied with the wide network in terms of validation accuracy but it's half the size and faster. Tables 2.1, 2.2, and 2.3 showcase the results of each experiment for easy comparison.

	Baseline (2 hidden layers)	**4 hidden layers**
Epochs	80	100
Converged at	~65 epochs	~60 epochs
Training time (s)	1954.3	2851.3
Time per epoch (s)	24.4	28.5
Training loss	0.005157	0.000285
Training accuracy	0.999960	1.000000
Validation accuracy	0.542000	0.558000
Trainable parameters	1,841,162	2,366,474

Table 2.1 - Baseline neural network with 2 and 4 hidden layers.

	4 hidden layers (1.b)	**10 hidden layers**
Epochs	100	80
Converged at	~60 epochs	Did not converge
Training time (s)	2851.3	3136.4
Time per epoch (s)	28.5	39.2
Training loss	0.000285	0.642339
Training accuracy	1.000000	0.755080
Validation accuracy	0.558000	0.434100
Trainable parameters	2,366,474	3,942,410

Table 2.2 - 4 hidden layers vs 10 hidden layers

	Baseline (ReLU, 512)	**Exp 1: Width (ReLU, 1024)**	**Exp 2: Sigmoid**	**Exp 3: Tanh**
Epochs	80	80	80	80
Converged at	~65 epochs	~57 epochs	Did not converge	~68 epochs
Training time (s)	1954.3	3301.8	2071.6	2088.3
Time per epoch (s)	24.4	41.3	25.9	26.1
Training loss	0.005157	0.005353	1.216184	0.019797
Training accuracy	0.999960	0.999960	0.581120	0.998260
Validation accuracy	0.542000	0.558100	0.494700	0.451500
Trainable parameters	1,841,162	4,206,602	1,841,162	1,841,162

Table 2.3 - Baseline vs wide vs Sigmoid vs Tanh